

Control Machine

A factory of agents on one computer. You talk to each agent in Telegram.
The computer does the work.

Author
Shivank.sharma@wipro.com

Index

Control Machine keeps a factory of agents on the platform and one computer they share. Each agent has its own Telegram chat. The Mac dials out, and one agent works at a time. These pages cover the product, the services it runs on, who it is for, and why the pause stays in the same chat. Start with the introduction for the loop: ask, act, pause, reply. Core Features and Infra is the Factory, the bot, Postgres, Chrome, and the live socket. The later pages say who gains from that arrangement, and the close puts the same loop in one place. The sample throughout is Office filling a MyWipro timesheet.

Introduction	3
A factory of agents, one computer, and a Telegram chat for each agent. The timesheet run is the example of a pause.	
Core Features and Infra	4
The factory, the bot for each agent, Postgres, Docker Chrome, and the JPEG socket. The diagram is on this page.	
Key Business Benefits	5
Work starts in the chat you already use, on the computer you already have. One session holds the Mac.	
Target Audience of Product	6
Someone who wants several agents on one Mac, reached from Telegram. A form, a draft, and a repo can be three agents.	
Conclusion	7
Ask in Telegram. One agent works on the Mac. You step in only for the step that is yours. The chat keeps the record.	

Introduction

Control Machine is a factory of agents that share one computer. Each agent has its own Telegram bot, and the person talks to that bot in one chat. The agent decides what to do. The computer carries it out.

The platform stays up when the Mac is off. The agent, its memory, its skills, and its activity stay readable. A connector on the Mac dials out. While that connection is quiet, the agent refuses machine work and says so in the chat. When the Mac is online, one agent holds it.

Office, Sales, and Coder are the first three. Office fills a timesheet and is the run that shows a pause. Sales works a pipeline in the browser. Coder reads the repo on the Mac and asks before it writes. Opening one in the Factory loads that agent: its prompt, its skills, its memory, and its tools.

A sensitive step stops. A sign-in, a code, or a submit is that kind of step. The agent waits. The person answers in the same Telegram chat, or opens the live desktop from the phone and finishes the step. The picture is a JPEG a few times a second on a signed socket. Taps and typing go back on that socket. The reply in the chat is what lets the same session continue.

The agent sees page text and screenshots. Passwords, the machine credential, bot tokens, and the keystrokes from the live desktop stay off the record.

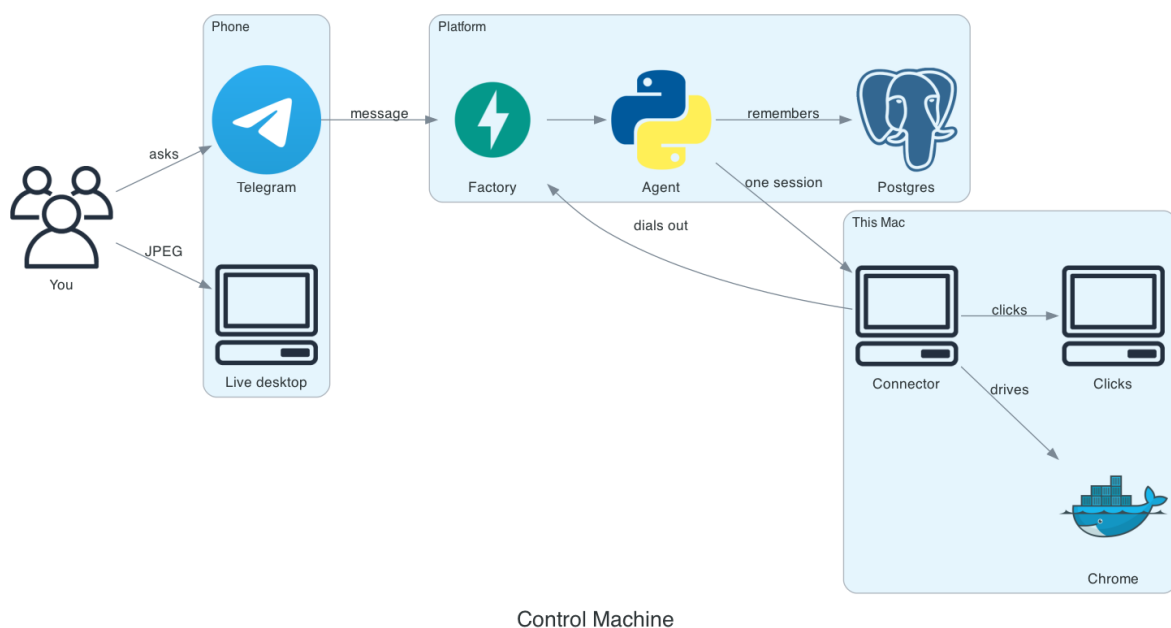
A timesheet run shows the loop. The person asks Office, in its chat, to fill this week's MyWipro efforts and to stop before submit. Office opens Chrome, reads the sign-in page, and pauses. The person signs in on the live desktop and replies in that chat. Office reads the page again, enters the hours, and stops once more before the form is sent.

Sales and Coder are created the same way. The Factory row saves the name and the job. Connecting the bot starts a poller for that token. The first message compiles the graph from that agent's prompt, skills, memory, and tools. The computer they reach is the one Mac, and only one of them holds it at a time.

Core Features and Infra

The working surface is the Factory. The services under it are the ones this computer already runs.

- Factory lists the agents on this Mac. New agent saves the name and the job, then the bot is connected and the graph is compiled from that row.
- One Telegram bot per agent. The poller for that bot hears the chat. The token stays outside the tables.
- The Deep Agent for a message uses that agent's prompt, skills, memory, and enabled tools. Browser and machine clicks start on. SAP stays off until the browser and the clicks cannot drive the screen.
- Postgres holds the agents, skills, memory, sessions, pauses, messages, and events. LangGraph checkpoints stay in their own tables.
- Docker Chrome is the browser. The agent reads the page and acts on element refs. The connector on the Mac dials out to the platform.
- The live desktop is JPEG frames on a signed socket, injected with Accessibility. The private noVNC view stays inside the browser container.



You ask in Telegram. The agent on the platform uses one session on this Mac.

Key Business Benefits

The gain is that each agent is a standing worker, and the computer stays one machine you already trust.

- A job starts as a message. The person does not sit down at the Mac to begin it, and the result comes back in the same chat.
- Each agent keeps its own skills and memory, so a timesheet routine and a pipeline routine do not share a conversation or a prompt.
- The platform keeps working while the Mac is asleep. The agent can still chat. Machine work waits until the connector dials out.
- A login or a submit stops in the chat. The person finishes that step on the phone, and the same session continues.
- The browser, the files, and the screen stay on this computer. The credential that lets the connector dial out stays on the computer too.

A repeated job becomes a skill on that agent. The next time the person asks, the agent follows the same instructions and stops in the same places. The first timesheet teaches the handoff. The skill lets Office start it again.

The computer runs one intent. If Office holds the Mac, Sales can still answer in its own chat and waits to open a session. When the connector drops, a pause that was already waiting stays waiting until the Mac dials out again.

The catalog grows by agent. A new skill is instructions on that agent. A new agent is another bot and another prompt, pointed at the same connector, the same Chrome, and the same socket. The way you ask does not change.

Target Audience of Product

Control Machine is for a person who already has the computer and the chat, and wants agents to take the screen work.

- Someone running one Mac for desk work, with a phone as the way in.
- A person who wants a separate agent for separate jobs, such as timesheets, a pipeline, and a repo, each with its own bot.
- An operator who must sign in or approve a submit themselves, then let the agent continue from the page already open.
- A team that can share the pattern while each person runs their own machine and their own bots.

The tools stay familiar. Telegram is the conversation. Chrome and the apps already on the Mac are the surface the agent operates. The Factory is where those agents are listed and opened.

The day looks like this. The person opens Telegram, picks the bot for the job, and writes the ask. If the Mac is on, that agent takes the session. If a page asks for a password, the chat receives the pause and a link to the screen. After the reply, the agent finishes and the chat holds the record.

It fits the person who files the same form every week, the person who drafts in a browser and wants a check before anything is sent, and the person who wants an agent to read a repo and stop before it writes. Each of those is a different agent. The computer under them is the same one. The Factory is the list they open. The chat is the only conversation each agent has.

Conclusion

Control Machine connects a factory of agents to one computer. You ask in that agent's Telegram chat. The agent decides. The Mac does the work when the connector is online, and one intent at a time.

The loop is the same for every agent. A message starts the session. If a sign-in or an approval appears, the agent waits and the phone can show the screen. The reply in the same chat continues the session. The live desktop writes nothing.

Office, Sales, and Coder are three workers on that pattern. What they keep — prompt, skills, memory, and tools — belongs to each of them. What they share is the connector, the browser, the JPEG socket, and the lock that lets only one session hold the Mac.

Use it when the work is on the screen and you are somewhere else. Ask the agent. Let it run what it can. Take the step that belongs to you. Read the result in the same chat.

What a run writes is the ask, one session, the events along the way, and a pause when it has to wait. The person's reply answers that pause. The live desktop writes nothing. Signing in does not put a password in the chat or in the activity.

The agents are the workers. The skill is the job an agent can repeat. The chat is where you ask and where the agent asks you back. The Mac is the one machine they share, and it joins the work only while its connector is dialed out. Several agents, one computer, and a phone for the step that has to be yours. Connecting a new agent adds a row, a bot, and a graph compiled from that agent's own prompt. The connector, Chrome, and the JPEG socket stay the ones already on this Mac.